

CS 6341 Robotics Homework 2

Instructor Yu Xiang

September 25, 2026

In this homework, write down your solutions for Problem 1 and finish the coding tasks in Problem 2. Upload your solutions, code, and required screenshots to eLearning. Our TA will check your solutions and run your scripts to verify them.

For Problem 2, continue using the same ROS 2 Jazzy Docker environment from Homework 1. However, you will create a new ROS 2 workspace for Homework 2 so that it does not interfere with your Homework 1 workspace.

Problem 1

(2 points)

Homogeneous Transformation Matrices. Exercise 3.18 in Lynch and Park, *Modern Robotics*.

Consider a robot arm mounted on a spacecraft as shown in Figure 1, in which frames are attached to the Earth $\{e\}$, a satellite $\{s\}$, the spacecraft $\{a\}$, and the robot arm $\{r\}$, respectively.

(1.1) Given T_{ea} , T_{ar} , and T_{es} , find T_{rs} .

(1.2) Suppose that the frame $\{s\}$ origin as seen from $\{e\}$ is $(1, 1, 1)$ and that

$$T_{er} = \begin{bmatrix} -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Write down the coordinates of the frame $\{s\}$ origin as seen from frame $\{r\}$.

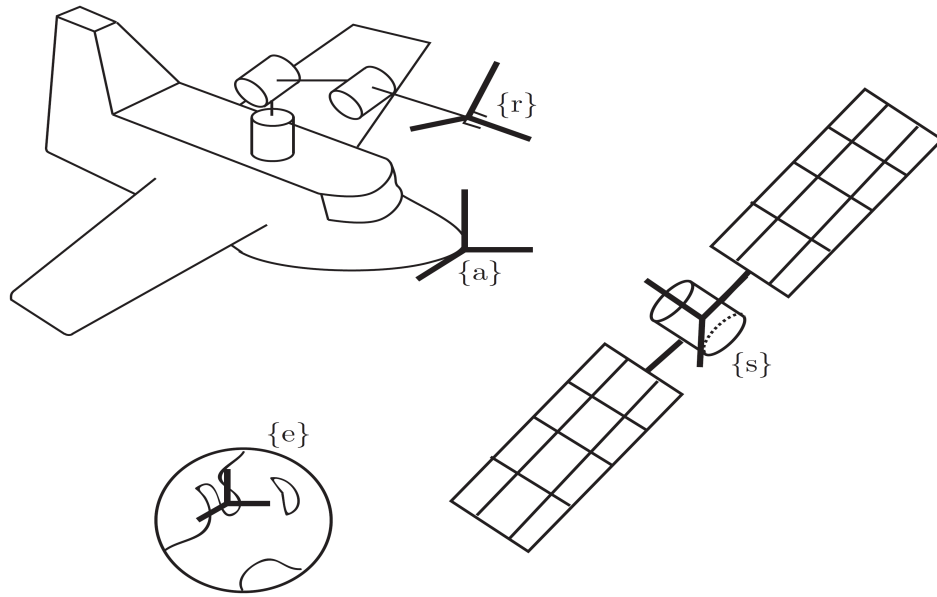


Figure 1: A robot arm mounted on a spacecraft.

Problem 2

(8 points)

ROS 2 programming and transformations.

In this problem, you will use homogeneous transformations and TF in ROS 2 and visualize the results using Gazebo and RViz. We will continue using **ROS 2 Jazzy Jalisco** and the Docker environment from Homework 1.

Refer to the ROS 2 Jazzy documentation if needed:

<https://docs.ros.org/en/jazzy/>

Before starting, launch the Docker container from Homework 1 and open the Ubuntu desktop at <http://127.0.0.1:6080/>. All commands below should be run in terminals inside the Docker Ubuntu desktop.

(2.1) Install and test Gazebo.

Open a terminal inside the Docker Ubuntu desktop and source ROS 2:

```
source /opt/ros/jazzy/setup.bash
```

Install the ROS 2 Gazebo packages:

```
sudo apt update
sudo apt install ros-jazzy-ros-gz
```

Test the Gazebo installation:

```
gz sim
```

A Gazebo window should open. Close Gazebo after confirming that it launches correctly.

(2.2) Create a new ROS 2 workspace for Homework 2.

To avoid conflicts with packages used in Homework 1, create a new ROS 2 workspace for this homework.

Create the workspace under your mounted data directory:

```
mkdir -p ~/data/ros2_ws_hw2/src
```

Your Homework 2 workspace should therefore be:

```
~/data/ros2_ws_hw2
```

and ROS 2 packages should be placed under:

```
~/data/ros2_ws_hw2/src
```

For Docker users, because ~/data is mounted from the host computer, your Homework 2 source code will also be stored on your Windows or Mac computer.

(2.3) Add the Fetch Gazebo simulator to your workspace.

Go to the src directory:

```
cd ~/data/ros2_ws_hw2/src
```

Clone the following repository using the master branch:

```
git clone -b master https://github.com/IRVLUTD/panda_gz_moveit2.git
```

This repository contains the Fetch robot description, MoveIt configuration, ROS 2 control configuration, and Gazebo launch files used in this homework.

Before installing dependencies, source ROS 2:

```
source /opt/ros/jazzy/setup.bash
```

Go to the root of your Homework 2 workspace:

```
cd ~/data/ros2_ws_hw2
```

Install package dependencies using rosdep:

```
rosdep install -i --from-paths src --rosdistro jazzy -y
```

Then build the workspace:

```
colcon build --symlink-install
```

Do not use `sudo colcon build`.

Make sure that the workspace builds successfully without errors.

After building, source the workspace:

```
source install/setup.bash
```

(2.4) Launch the Fetch Gazebo simulation.

Before launching the simulation, make sure both ROS 2 and your Homework 2 workspace are sourced:

```
source /opt/ros/jazzy/setup.bash
cd ~/data/ros2_ws_hw2
source install/setup.bash
```

Add the Fetch model directories to the Gazebo resource path:

```
export GZ_SIM_RESOURCE_PATH=$GZ_SIM_RESOURCE_PATH:$(ros2 pkg prefix
  fetch_moveit_config)/share:$(ros2 pkg prefix fetch_description)/share
```

Launch the Fetch simulation:

```
ros2 launch panda fetch.launch.py
```

The launch file should open both Gazebo and RViz.

If needed, you can change the Gazebo camera viewpoint using (open another terminator window and source your workspace):

```
gz service -s /gui/move_to/pose \
--reqtype gz.msgs.GUICamera \
--reptype gz.msgs.Boolean \
--timeout 5000 \
--req 'pose:{position:{x:3,y:0,z:2} orientation:{x:0,y:0,z:1.0,w:0.0}}'
```

If everything is launched correctly, you should see interfaces similar to Figure 2.

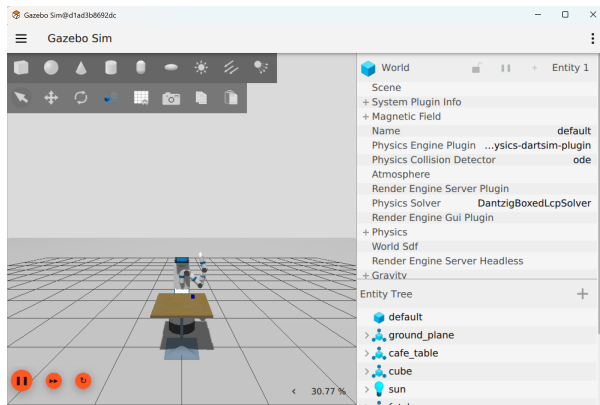
Capture a screenshot showing the Fetch robot in Gazebo and include it in your homework submission.

(2.5) Visualize the TF tree in RViz.

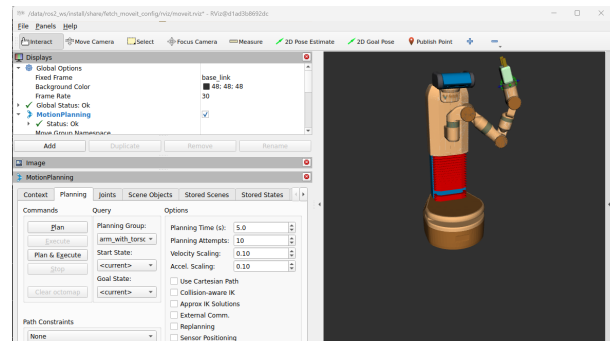
ROS 2 uses TF to represent coordinate frames and transformations between robot links.

In the RViz window:

- Click the **Add** button.
- Select **TF**.
- Add the TF display.



Gazebo interface



RViz interface

Figure 2: Gazebo and RViz after launching the Fetch simulation.

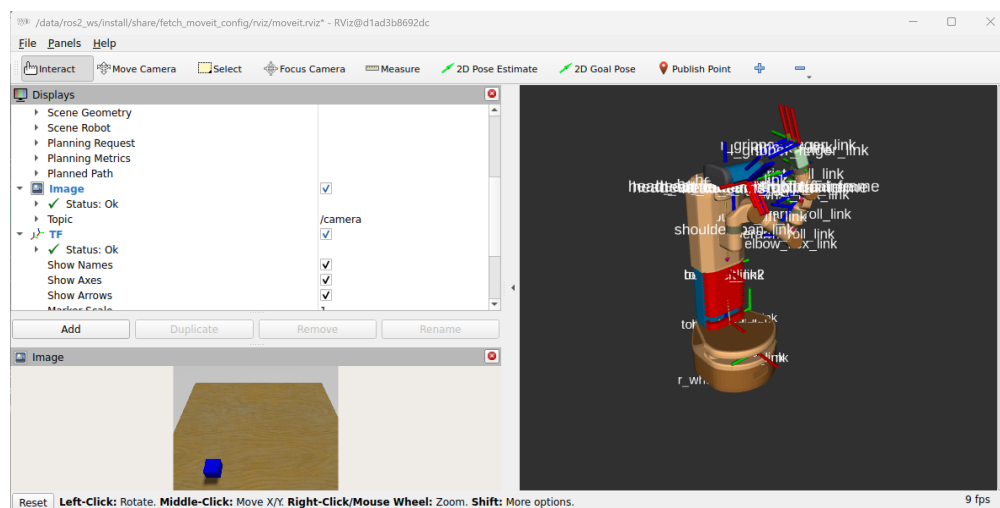


Figure 3: TF frames visualized in RViz.

You should now see the coordinate frames associated with different links of the Fetch robot, similar to Figure 3.

Spend some time examining the TF tree and how the coordinate frames are connected.

Capture a screenshot showing the TF frames in RViz and include it in your homework submission.

(2.6) Compute the pose of the demo cube in the Fetch base_link frame.

Download the file `visualize_block_pose.py` from eLearning and save it inside your mounted data directory. For example:

```
~/data/visualize_block_pose.py
```

The Python script obtains:

- the pose of the demo cube in the Gazebo world frame, and
- the pose of the Fetch robot base in the Gazebo world frame.

Your task is to compute the pose of the cube relative to the Fetch base_link frame using homogeneous transformation matrices.

Finish the implementation of the callback function:

```
def cb():
```

```
in visualize_block_pose.py.
```

Your implementation should use the two poses provided by Gazebo to compute the correct transformation from base_link to the cube.

The script publishes the resulting transformation to ROS 2 TF so that it can be visualized in RViz.

Install the transforms3d Python package if needed:

```
pip3 install --upgrade transforms3d --break-system-packages
```

Keep the Fetch Gazebo simulation running. Open another terminal inside the Docker desktop and source ROS 2 and your Homework 2 workspace:

```
source /opt/ros/jazzy/setup.bash
cd ~/data/ros2_ws_hw2
source install/setup.bash
```

Then run:

```
python3 ~/data/visualize_block_pose.py
```

If your transformation is correct, the cube coordinate frame should appear in RViz at the correct location, similar to Figure 4.

Capture a screenshot showing the computed cube pose in RViz and include it in your homework submission.

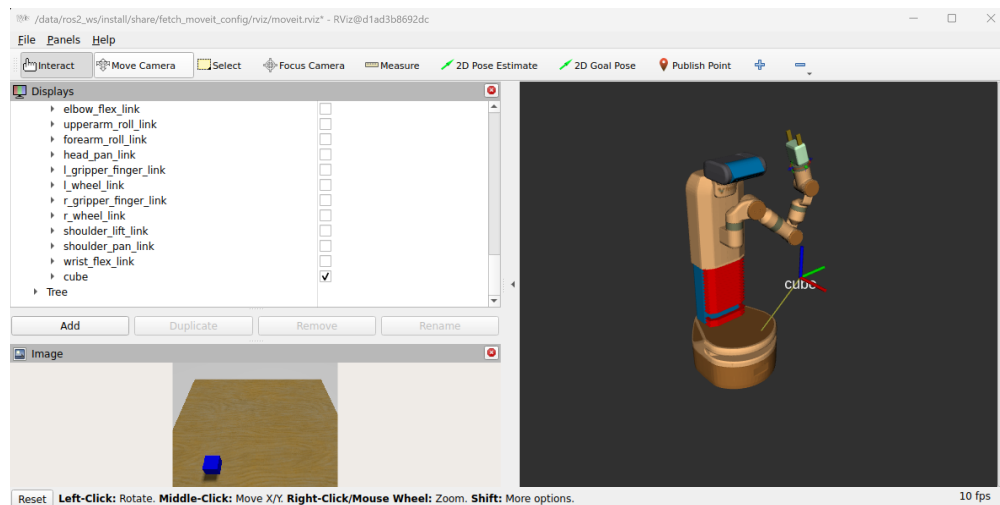


Figure 4: The computed cube pose visualized in RViz.

Submission guideline: Submit the following items to eLearning: (1) your written solution to Problem 1, (2) your completed `visualize_block_pose.py` file, and (3) screenshots for Problems 2.4, 2.5, and 2.6. Make sure your Python script runs using the ROS 2 Jazzy Homework 2 workspace located at `~/data/ros2_ws_hw2`.